

Описание вычислительного узла:

Наименование и краткая характеристика CPU:

Architecture: x86_64
CPU op-mode(s): 32-bit, 64-bit
Address sizes: 46 bits physical, 48 bits virtual
Byte Order: Little Endian
CPU(s): 80
On-line CPU(s) list: 0-79
Vendor ID: GenuineIntel
Model name: Intel(R) Xeon(R) Gold 6248 CPU @ 2.50GHz
CPU family: 6
Model: 85
Thread(s) per core: 2
Core(s) per socket: 20
Socket(s): 2
Stepping: 7
CPU max MHz: 3900.0000
CPU min MHz: 1000.0000
BogoMIPS: 5000.00
Flags: fpu vme de pse tsc msr pae mce cx8 apic sep mtrr pge mca cmov pat pse36 clflush dts acpi mmx fxsr sse sse2 ss ht tm pbe syscall nx pdpe1gb rdtscp lm constant_tsc art arch_perfmon pebs bts rep_good nopl xtopology nonstop_tsc cpuid aperfmperf pni pclmulqdq dtes64 monitor ds_cpl smx est tm2 ssse3 sdbg fma cx16 xtpr pdcm pcid dca sse4_1 sse4_2 x2apic movbe popcnt tsc_deadline_timer aes xsave avx f16c rdrand lahf_lm abm 3dnowprefetch cpuid_fault epb cat_l3 cdp_l3 invpcid_single intel_ppin ssbd mba ibrs ibpb stibp ibrs_enhanced fsgsbase tsc_adjust bmi1 avx2 smep bmi2 erms invpcid cqm mpx rdt_a avx51

2f avx512dq rdseed adx smap clflushopt clwb intel_
pt avx512cd avx512bw avx512vl xsaveopt xsavec xget
bv1 xsaves cqm_llc cqm_occup_llc cqm_mbm_total cqm_
_mbm_local dtherm ida arat pln pts hwp hwp_act_win
dow hwp_pkg_req pku ospke avx512_vnni md_clear flu
sh_l1d arch_capabilities

Caches (sum of all):

L1d: 1.3 MiB (40 instances)
L1i: 1.3 MiB (40 instances)
L2: 40 MiB (40 instances)
L3: 55 MiB (2 instances)

NUMA:

NUMA node(s): 2
NUMA node0 CPU(s): 0-19,40-59
NUMA node1 CPU(s): 20-39,60-79

Vulnerabilities:

Gather data sampling: Mitigation; Microcode
Itlb multihit: KVM: Mitigation: VMX unsupported
L1tf: Not affected
Mds: Not affected
Meltdown: Not affected
Mmio stale data: Mitigation; Clear CPU buffers; SMT vulnerable
Reg file data sampling: Not affected
Retbleed: Mitigation; Enhanced IBRS
Spec rstack overflow: Not affected
Spec store bypass: Mitigation; Speculative Store Bypass disabled via
prctl and seccomp
Spectre v1: Mitigation; usercopy/swapgs barriers and __user po
inter sanitization
Spectre v2: Mitigation; Enhanced / Automatic IBRS; IBPB condit
ional; RSB filling; PBRBSB-eIBRS SW sequence; BHI S
W loop, KVM SW loop

Srbds: Not affected

Tsx async abort: Mitigation; TSX disabled

Наименование сервера: ProLiant XL270d Gen10

NUMA node:

available: 2 nodes (0-1)

node 0 cpus: 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 40 41 42 43 44 45
46 47 48 49 50 51 52 53 54 55 56 57 58 59

node 0 size: 385636 MB

node 0 free: 71537 MB

node 1 cpus: 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 60
61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79

node 1 size: 387008 MB

node 1 free: 105247 MB

node distances:

node 0 1

0: 10 21

1: 21 10

Операционная система:

PRETTY_NAME="Ubuntu 22.04.5 LTS"

NAME="Ubuntu"

VERSION_ID="22.04"

VERSION="22.04.5 LTS (Jammy Jellyfish)"

VERSION_CODENAME=jammy

ID=ubuntu

ID_LIKE=debian

HOME_URL="https://www.ubuntu.com/"

SUPPORT_URL="https://help.ubuntu.com/"

BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"

PRIVACY_POLICY_URL="https://www.ubuntu.com/legal/terms-and-policies/privacy-policy"

UBUNTU_CODENAME=jammy

Выводы о масштабируемости на основе графиков:

Задание 1 (Умножение матрицы на вектор)

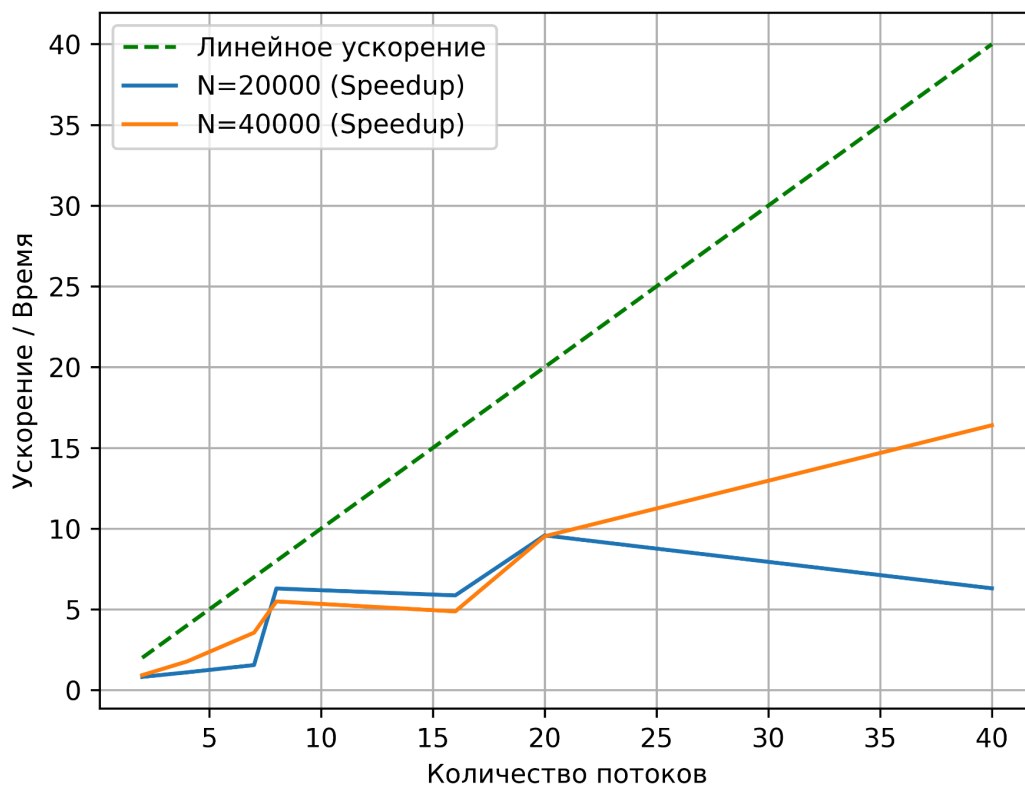


График показывает, что для матриц размером 20000x20000 и 40000x40000 в зависимости от количества потоков, наблюдается следующее:

- Ускорение близко к линейному при малом количестве потоков (до 8), что свидетельствует о хорошей масштабируемости.
- При увеличении числа потоков (более 16) рост ускорения замедляется, особенно для матрицы 20000x20000, что может быть связано с накладными расходами на управление потоками или ограничением пропускной способности памяти.
- Для матрицы 40000x40000 масштабируемость лучше, так как размер задачи компенсирует накладные расходы.

Вывод: Программа демонстрирует хорошую масштабируемость до 8-16 потоков, после чего эффективность снижается. Увеличение размера задачи улучшает масштабируемость.

Задание 2 (Численное интегрирование)

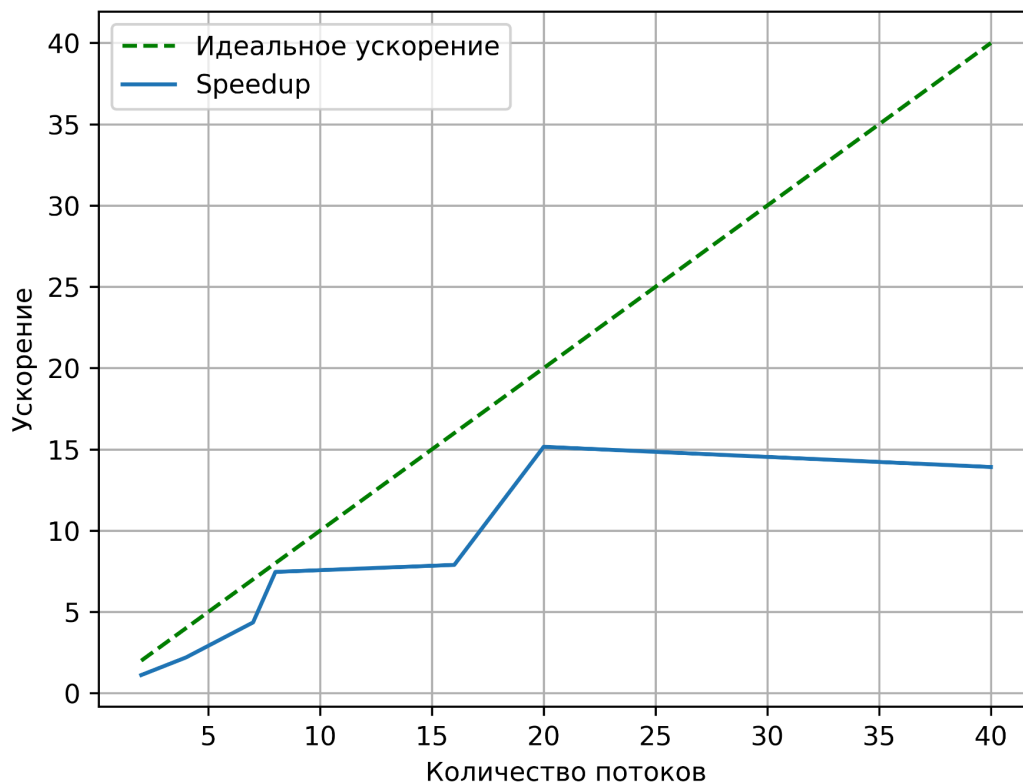


График показывает:

- Ускорение близко к идеальному при малом числе потоков (до 8), что указывает на эффективное распараллеливание.
- При большем числе потоков (20-40) наблюдается отклонение от идеального ускорения, вероятно, из-за contention на атомарных операциях или неравномерной нагрузки.

Вывод: Метод хорошо масштабируется до 8 потоков, но дальнейшее увеличение потоков не дает пропорционального ускорения из-за накладных расходов.

Задание 3 (Метод простой итерации)

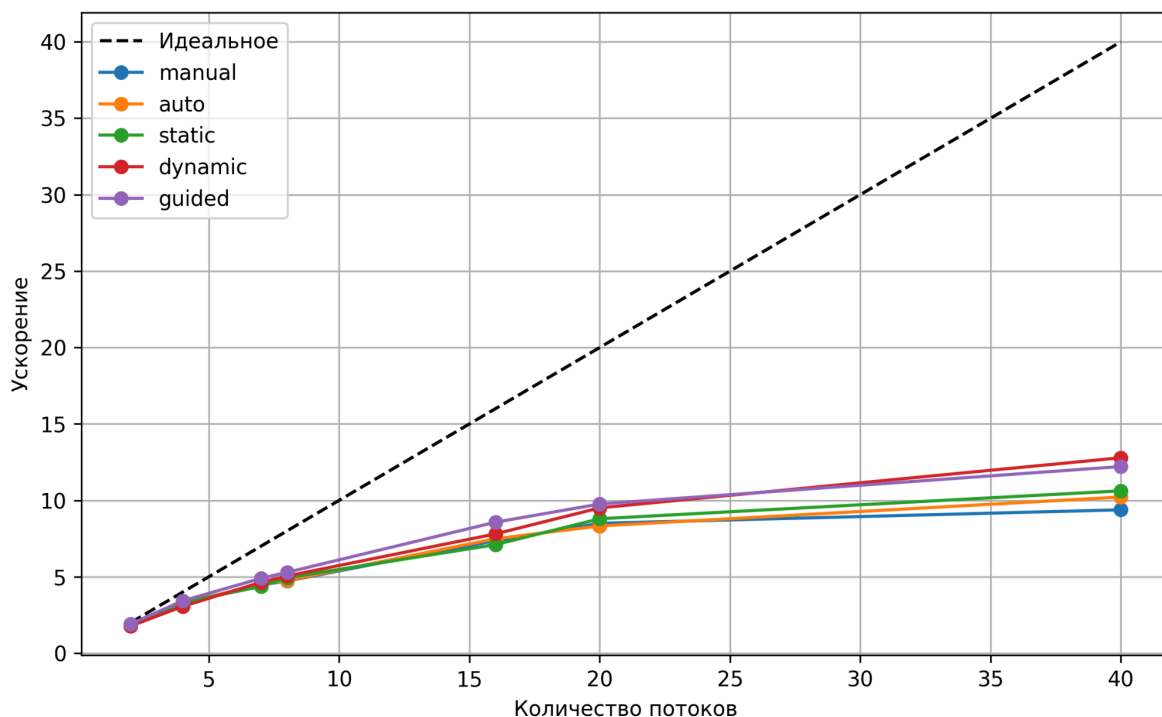


График показывает:

- Стратегия `static` показывает наилучшую масштабируемость, близкую к идеальной, так как нагрузка равномерна.
- Стратегии `dynamic` и `guided` менее эффективны из-за накладных расходов на динамическое распределение итераций.
- При большом числе потоков (20-40) все стратегии отклоняются от идеала, что связано с ограничениями памяти или синхронизации.

Вывод: Для данного метода лучше использовать `static` распараллеливание. Масштабируемость хорошая до 16 потоков, после чего эффективность падает.

Общий вывод:

Все три задачи демонстрируют хорошую масштабируемость при малом числе потоков (до 8-16), но при дальнейшем увеличении потоков эффективность снижается из-за накладных расходов и ограничений памяти. Наибольшая масштабируемость наблюдается для задач с большим объемом вычислений (матрица 40000x40000) и равномерной нагрузкой (статическое распараллеливание).